

QUICK REFERENCE GUIDE

Fluoride Adsorption Hybrid Modeling - Phase 1 Execution

ONE-LINER START

```
cd fluoride-adsorption-hybrid && source venv/bin/activate && python src/doe_design.py && python
```

QUICK CHECKLIST

Before Starting

- ☐ Python 3.7+ installed
- ☐ Virtual environment created
- ☐ Dependencies installed (`pip install pyDOE2 pandas numpy scipy matplotlib`)
- ☐ Project folder created

Step 1: Setup (30 min)

- ☐ Create project directory: `mkdir fluoride-adsorption-hybrid`
- ☐ Create folder structure: `mkdir -p {data,src,notebooks,models,app,reports,output}`
- ☐ Create venv: `python -m venv venv`
- ☐ Activate venv: `source venv/bin/activate`
- ☐ Install deps: `pip install pyDOE2 pandas numpy scipy matplotlib seaborn jupyter`

Step 2: DoE Generation (5 min)

- ☐ Run: `python src/doe_design.py`
- ☐ Verify: `data/doe_matrix.csv` exists
- ☐ Check: 48 rows $\times$ 8 columns

Step 3: Simulation (2 min)

- ☐ Run: `python src/simulate_adsorption.py`
- ☐ Verify: `data/dataset_simulated.csv` exists
- ☐ Check: 48 rows $\times$ 7 columns
- ☐ Verify: Efficiency 15-97%, Capacity 0.5-8.5 mg/g

Step 4: Validation (10 min)

- ☐ Plot: `python notebooks/01_data_exploration.py`
- ☐ Check: `output/01_data_exploration.png` created
- ☐ Review: pH bell curve, time saturation, temp increase, flow decrease

Ready for Phase 2

- ☐ All files created and verified

- ☐ Data explores show expected trends
- ☐ Proceed to Langmuir fitting

FILE LOCATIONS

File	Purpose	Status
src/doe_design.py	Generate 48-run CCD	Create
src/simulate_adsorption.py	Physics-based simulation	Create
data/doe_matrix.csv	Experimental design	Generate
data/dataset_simulated.csv	Simulation results	Generate
output/01_data_exploration.png	Validation plots	Generate

COMMON COMMANDS

Environment

```
# Create
python -m venv venv

# Activate (macOS/Linux)
source venv/bin/activate

# Activate (Windows)
venv\Scripts\activate

# Install all
pip install pyDOE2 pandas numpy scipy matplotlib seaborn jupyter xgboost scikit-learn

# Save requirements
pip freeze > requirements.txt
```

Execution

```
# Generate DoE
python src/doe_design.py

# Run simulation
python src/simulate_adsorption.py

# Create plots
python notebooks/01_data_exploration.py

# Quick stats
python -c "import pandas as pd; print(pd.read_csv('data/dataset_simulated.csv').describe())"
```

Verification

```
# List all data files
ls -lh data/*.csv

# Count rows
wc -l data/*.csv

# View first rows
head -5 data/doe_matrix.csv
head -5 data/dataset_simulated.csv

# Summary statistics
python << 'EOF'
import pandas as pd
df = pd.read_csv('data/dataset_simulated.csv')
print(df.describe())
EOF
```

EXPECTED OUTPUTS

DoE Matrix (data/doe_matrix.csv)

```
Run,pH,C0,Time,Temp,Flow,Order
1,3.0,1.0,10,20,0.5,25
2,6.0,5.5,65,30,1.25,12
3,9.0,10.0,120,40,2.0,8
...
48,6.0,5.5,65,30,1.25,44
```

- **Size:** 48 rows $\times$ 8 columns
- **Factors:** pH, C0, Time, Temp, Flow (all in physical units)

Dataset (data/dataset_simulated.csv)

```
Run,pH,C0_mg_L,Time_min,Temp_C,Flow_L_min,Efficiency_percent,Capacity_mg_g
1,3.0,1.0,10,20,0.5,42.3,0.42
2,6.0,5.5,65,30,1.25,89.2,4.90
...
48,6.0,5.5,65,30,1.25,88.7,4.87
```

- **Size:** 48 rows $\times$ 7 columns
- **Responses:** Efficiency (%), Capacity (mg/g)

Validation Statistics

```
=====
SIMULATION VALIDATION
=====
```

pH Dependence:

pH 3.0: 45.23% (low)
pH 6.5: 89.12% (PEAK)
pH 9.0: 42.67% (low)

Time Dependence:

t=10 min: 28.90% (early)
t=120 min: 94.78% (saturation)

Temperature Effect:

T=20°C: 76.45%
T=40°C: 85.67% (increase)

Flow Rate Effect:

Q=0.5 L/min: 88.45%
Q=2.0 L/min: 62.34% (decrease)

TIME ESTIMATES

Task	Time
Setup environment	10 min
DoE generation	5 min
Simulation	2 min
Data validation	5 min
Exploration plots	10 min
Total Phase 1	~30-45 min

SUCCESS METRICS

Phase 1.1 (DoE)

- ☐ `doe_matrix.csv` has 48 rows
- ☐ All 5 factors present (pH, C0, Time, Temp, Flow)
- ☐ Values within expected ranges:
 - pH: 3.0 - 9.0
 - C0: 1.0 - 10.0 mg/L
 - Time: 10 - 120 min
 - Temp: 20 - 40 °C
 - Flow: 0.5 - 2.0 L/min

Phase 1.2 (Simulation)

- ☐ `dataset_simulated.csv` has 48 rows
- ☐ Efficiency: 15-97%

- ☐ Capacity: 0.5-8.5 mg/g
- ☐ No NaN or negative values
- ☐ Data passes validation checks

Phase 1.3 (Validation)

- ☐ pH curve peaks at 6.5
 - ☐ Time shows saturation at 120 min
 - ☐ Temperature shows positive effect
 - ☐ Flow rate shows negative effect
 - ☐ Distribution appears normal with realistic noise
-

TROUBLESHOOTING

Problem: ModuleNotFoundError: No module named 'pyDOE2'

```
pip install pyDOE2
pip list | grep pyDOE
```

Problem: FileNotFoundError: 'data/doe_matrix.csv'

```
# Make sure data folder exists
mkdir -p data

# Make sure doe_design.py ran first
python src/doe_design.py
```

Problem: Data not showing expected trends

```
# Check noise level (default ±5%)
# Look at simulation script validate_simulation() output
# If issues, increase center points in DoE:
# In doe_design.py: design = ccdesign(n=5, center=8, face='face')
```

Problem: Negative efficiency values

```
# This shouldn't happen (code clips to [0, 100])
# If you see it, check noise_level setting:
# Default: 0.05 (5%) - should be fine
```

FILE TEMPLATES

src/doe_design.py

```
from pyDOE2 import ccdesign
import pandas as pd
import numpy as np

# Define factors
FACTORS = {
    'pH': {'low': 3, 'center': 6, 'high': 9},
    'CO': {'low': 1, 'center': 5.5, 'high': 10},
    'Time': {'low': 10, 'center': 65, 'high': 120},
    'Temp': {'low': 20, 'center': 30, 'high': 40},
    'Flow': {'low': 0.5, 'center': 1.25, 'high': 2.0}
}

# Generate CCD
design = ccdesign(n=5, center=6, face='face')

# Scale to physical units
# (see full script for details)

# Save
df.to_csv('data/doe_matrix.csv', index=False)
```

src/simulate_adsorption.py

```
class FluorideAdsorptionSimulator:
    def __init__(self):
        self.qmax_ref = 8.5          # mg/g
        self.KL_ref = 0.12           # L/mg
        self.E_a = 20.0              # kJ/mol
        self.pH_opt = 6.5            # optimal pH

    def langmuir_equilibrium(self, C_e, qmax, KL):
        return (qmax * KL * C_e) / (1 + KL * C_e)

    def temperature_correction(self, T):
        # Arrhenius correction

    def pH_effect(self, pH):
        # Gaussian bell curve

    def pseudo_second_order_kinetics(self, q_e, t):
        # PSO model

    def simulate_single_run(self, pH, CO, t, T, Q):
```

REFERENCE PARAMETERS

Langmuir Constants

Parameter	Value	Unit	Source
qmax	8.5	mg/g	Talat et al. 2018
KL (25°C)	0.12	L/mg	Literature consensus
Ea	20	kJ/mol	Thermodynamic studies

Operating Ranges (Literature-Validated)

Parameter	Low	Center	High	Unit
pH	3.0	6.0	9.0	-
C	1.0	5.5	10.0	mg/L
Time	10	65	120	min
Temperature	20	30	40	°C
Flow	0.5	1.25	2.0	L/min

Kinetic Parameters

Parameter	Value	Unit	Notes
k	0.05	g/(mg · min)	Pseudo-2nd-order
t_eq	240	min	Equilibrium time
pH_	1.5	-	pH bell curve width

NEXT STEPS AFTER PHASE 1

- Phase 2 (Langmuir Fitting)**
 - Fit SSL and DSL models
 - Extract qmax, KL
 - Calculate R², RMSE
- Phase 3 (Residual Analysis)**
 - Analyze residual patterns
 - Identify ML features
 - Check randomness
- Phase 4 (ML Training)**
 - Train RF, XGBoost, MLP
 - Cross-validation
 - Feature importance

4. **Phase 5 (Hybrid Model)**
 - Combine chemical + ML
 - Compare all approaches
 - Validation
 5. **Phase 6-8 (Dashboard + Report)**
 - Streamlit interface
 - Documentation
 - Final report
-

TIPS & TRICKS

Faster Execution

```
# Run both scripts in sequence  
python src/doe_design.py && python src/simulate_adsorption.py
```

Interactive Exploration

```
# Use Jupyter for interactive work  
jupyter notebook notebooks/01_data_exploration.py
```

Parallel Processing (For future phases)

```
from joblib import Parallel, delayed  
# Use for hyperparameter tuning in Phase 4
```

Save & Share

```
# Create archive  
tar -czf fluoride-project.tar.gz fluoride-adsorption-hybrid/  
  
# Or zip  
zip -r fluoride-project.zip fluoride-adsorption-hybrid/
```

SUPPORT

If you encounter issues:

1. Check **Troubleshooting** section above
 2. Verify **dependencies**: `pip list`
 3. Check **file permissions**: `ls -la data/`
 4. Review **script comments** (well-documented)
 5. Validate **data**: `python -c "import pandas as pd; print(pd.read_csv('data/dataset_simulated.csv'))"`
-

LEARNING OUTCOMES

By completing Phase 1, you will understand: - Central Composite Design (CCD) - Physics-based simulation - Langmuir isotherm mechanics - Kinetic modeling (PSO) - Temperature effects (Arrhenius) - pH dependence - Data generation & validation

NOTES

- **Total execution time:** ~45 minutes
 - **No lab equipment needed:** Entirely computational
 - **Fully reproducible:** Same results every time
 - **Ready for Phase 2:** After completion, move to Langmuir fitting
-

Last Updated: May 3, 2026

Status: Ready for execution

Next Phase: Phase 2 - Langmuir Model Fitting

Ready? Start with: `python src/doe_design.py`